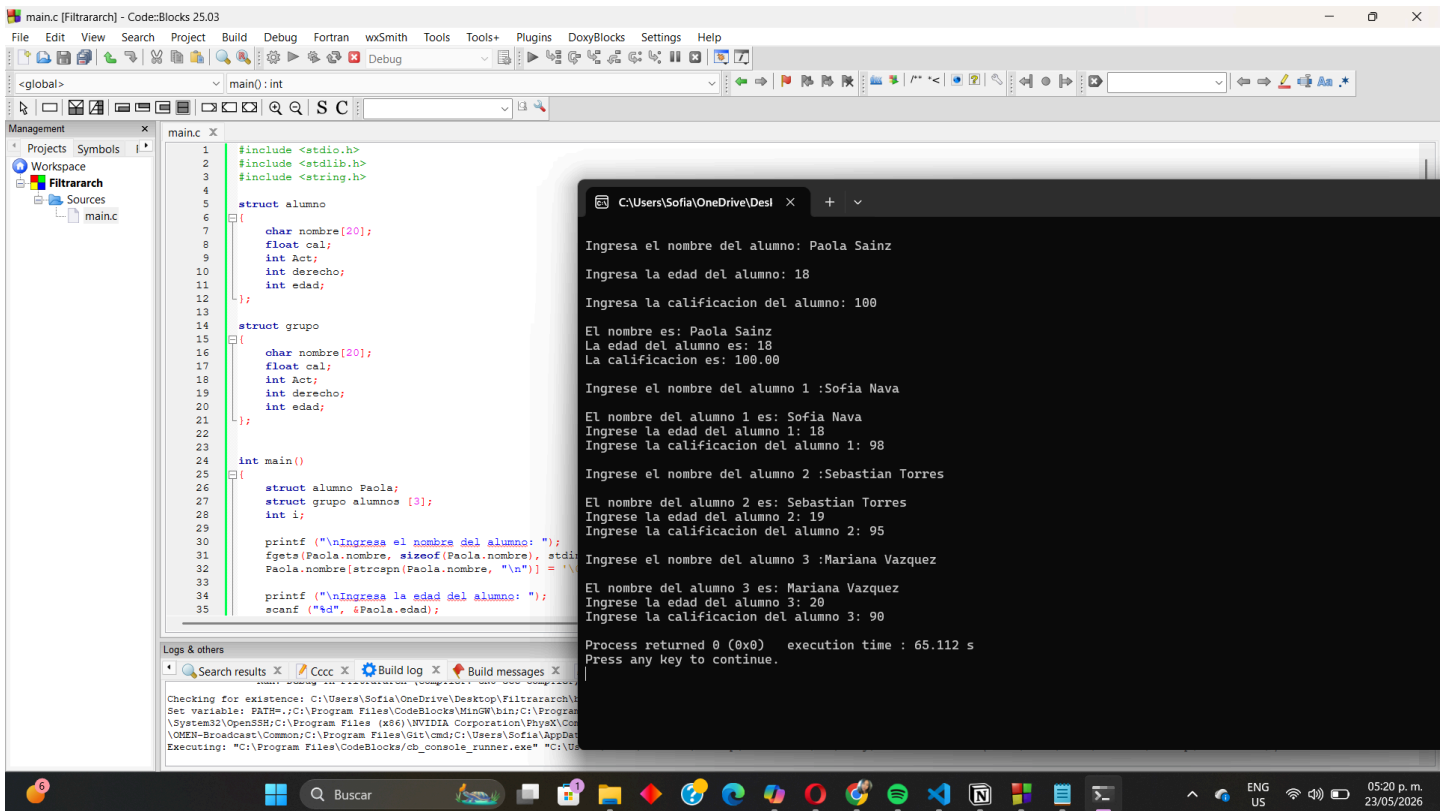


Structs y Union - Documentación

/////Capturas de funcionamientos de los códigos

- Código structs.c



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct alumno
{
    char nombre[20];
    float cal;
    int Act;
    int derecho;
    int edad;
};

struct grupo
{
    char nombre[20];
    float cal;
    int Act;
    int derecho;
    int edad;
};

int main()
{
    struct alumno Paola;
    struct grupo alumnos [3];
    int i;

    printf ("\nIngresar el nombre del alumno: ");
    fgets (Paola.nombre, sizeof (Paola.nombre), stdin);
    Paola.nombre[strlen (Paola.nombre)] = '\0';

    printf ("\nIngresar la edad del alumno: ");
    scanf ("%d", &Paola.edad);

    Ingrese el nombre del alumno: Paola Sainz
    Ingrese la edad del alumno: 18
    Ingrese la calificación del alumno: 100

    El nombre es: Paola Sainz
    La edad del alumno es: 18
    La calificación es: 100.00

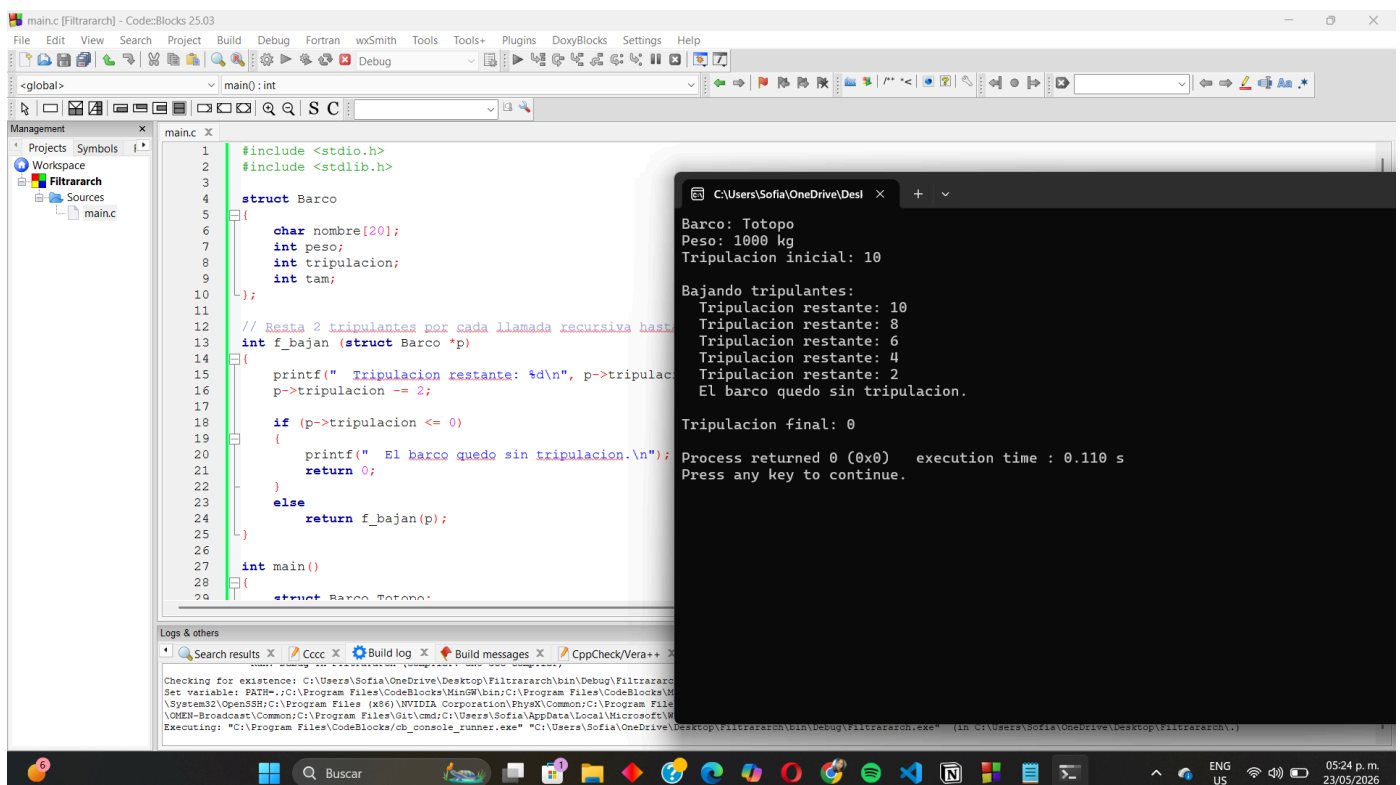
    Ingrese el nombre del alumno 1 :Sofia Nava
    El nombre del alumno 1 es: Sofia Nava
    Ingrese la edad del alumno 1: 18
    Ingrese la calificación del alumno 1: 98

    Ingrese el nombre del alumno 2 :Sebastian Torres
    El nombre del alumno 2 es: Sebastian Torres
    Ingrese la edad del alumno 2: 19
    Ingrese la calificación del alumno 2: 95

    Ingrese el nombre del alumno 3 :Mariana Vazquez
    El nombre del alumno 3 es: Mariana Vazquez
    Ingrese la edad del alumno 3: 20
    Ingrese la calificación del alumno 3: 90

    Process returned 0 (0x0) execution time : 65.112 s
    Press any key to continue.
```

- Código repaso_structs



```
#include <stdio.h>
#include <stdlib.h>

struct Barco
{
    char nombre[20];
    int peso;
    int tripulacion;
    int tam;
};

// Resta 2 tripulantes por cada llamada recursiva hasta
int f_bajan (struct Barco *p)
{
    printf (" Tripulacion restante: %d\n", p->tripulacion);
    p->tripulacion -= 2;

    if (p->tripulacion <= 0)
    {
        printf (" El barco quedo sin tripulacion.\n");
        return 0;
    }
    else
        return f_bajan(p);
}

int main()
{
    struct Barco Totopo;

    Barco: Totopo
    Peso: 1000 kg
    Tripulacion inicial: 10

    Bajando tripulantes:
    Tripulacion restante: 10
    Tripulacion restante: 8
    Tripulacion restante: 6
    Tripulacion restante: 4
    Tripulacion restante: 2
    El barco quedo sin tripulacion.

    Tripulacion final: 0

    Process returned 0 (0x0) execution time : 0.110 s
    Press any key to continue.
```

- Código structs_punteros_memoria.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 struct Barco
6 {
7     char nombre[20];
8     char tipo[20];
9     int tripulacion;
10    float carga_toneladas;
11};
12
13 int main()
14 {
15     // Utilizando malloc
16     // Reserva memoria para un solo barco dinamico
17     printf("---1. Malloc ---\n");
18
19     struct Barco *p_barco;
20     p_barco = (struct Barco*)malloc(sizeof(struct Barco));
21
22     strcpy(p_barco->nombre, "Aqua");
23     strcpy(p_barco->tipo, "Carguero");
24     p_barco->tripulacion = 10;
25     p_barco->carga_toneladas = 500.5;
26
27     printf("Barco: %s\n", p_barco->nombre);
28     printf("Tipo: %s\n", p_barco->tipo);
29     printf("Tripulacion: %d\n", p_barco->tripulacion);
30     printf("Carga: %.1f toneladas\n", p_barco->carga_toneladas);
31
32     // Utilizando calloc
33     // Reserva una flota de 3 barcos, todos inicializados en 0
34     printf("\n--- 2. Calloc ---\n");
35 }

```

```

---1. Malloc ---
Barco: Aqua
Tipo: Carguero
Tripulacion: 10
Carga: 500.5 toneladas

--- 2. Calloc ---
Flota inicial (3 barcos vacios):
Barco 1: nombre='', tripulacion=0, carga=0.0
Barco 2: nombre='', tripulacion=0, carga=0.0
Barco 3: nombre='', tripulacion=0, carga=0.0

--- 3. Realloc ---
Flota ampliada (5 barcos):
Barco 1: Sol | Tripulacion: 20 | Carga: 300.0 ton
Barco 2: Mar | Tripulacion: 15 | Carga: 450.0 ton
Barco 3: Costa | Tripulacion: 12 | Carga: 200.0 ton
Barco 4: Oceano | Tripulacion: 30 | Carga: 800.0 ton
Barco 5: Arena | Tripulacion: 25 | Carga: 650.0 ton

--- 4. Free ---
Memoria liberada. Fin del programa.

Process returned 0 (0x0) execution time : 0.069 s
Press any key to continue.

```

- Código struct_y_union

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // Struct: cada campo tiene su propio espacio de memoria
6 struct Barco
7 {
8     int codigo;
9     float carga;
10    char nombre[20];
11};
12
13 union IdentificadorBarco
14 {
15     int codigo;
16     float carga;
17     char nombre[20];
18};
19
20 int f_bajan (struct Barco *p)
21 {
22     printf(" Tripulacion restante: %d\n", p->codigo);
23     p->codigo -= 2;
24
25     if (p->codigo <= 0)
26     {

```

```

--- 1. Tamano en memoria ---
Size of struct: 28 bytes
Size of union: 20 bytes

--- 2. Comportamiento del Struct ---
Codigo: 10
Carga: 500.5 toneladas
Nombre: Totopo

--- 3. Funcion recursiva (bajan tripulantes) ---
Tripulacion inicial: 10
Tripulacion restante: 10
Tripulacion restante: 8
Tripulacion restante: 6
Tripulacion restante: 4
Tripulacion restante: 2
El barco quedo sin tripulacion.
Tripulacion final: 0

--- 4. Comportamiento del Union ---
Codigo (corrompido): 1869901652
Carga (corrompida): 75648929841169981657436913664.00
Nombre (correcto): Totopo

--- 5. Uso correcto del Union ---
Codigo guardado y leído: 202
Carga guardada y leída: 750.0
Nombre guardado y leído: Kraken

Process returned 0 (0x0) execution time : 0.102 s
Press any key to continue.

```